

you're using numeric suffixes for a series of variables, consider replacing them with a data structure, such as a list or dictionary.

Classes That Should Just Be Functions or Modules

Programmers who use languages such as Java are used to creating classes to organize their program's code. For example, let's look at this example `Dice` class, which has a `roll()` method:

```
>>> import random
>>> class Dice:
...     def __init__(self, sides=6):
...         self.sides = sides
...     def roll(self):
...         return random.randint(1, self.sides)
...
>>> d = Dice()
>>> print('You rolled a', d.roll())
You rolled a 1
```

This might seem like well-organized code, but think about what our actual needs are: a random number between 1 and 6. We could replace this entire class with a simple function call:

```
>>> print('You rolled a', random.randint(1, 6))
You rolled a 6
```

Compared to other languages, Python uses a casual approach to organizing code, because its code isn't required to exist in a class or other boilerplate structure. If you find you're creating objects simply to make a single function call, or if you're writing classes that contain only static methods, these are code smells that indicate you might be better off writing functions instead.

In Python, we use modules rather than classes to group functions together. Because classes must be in a module anyway, putting this code in classes just adds an unnecessary layer of organization to your code. Chapters 15 through 17 discuss these object-oriented design principles in